I2C RTL Design + Full UVM Verification — Complete Masterclass

Every thought, every decision, every line of code explained from scratch

> **Who this is for:** You know SystemVerilog basics. You have heard of UVM. You want to understand not just *what* the code does but *why* every single decision was made — from the first architectural thought to the last assertion. Read this like a senior engineer is sitting next to you explaining everything out loud.

TABLE OF CONTENTS

1. Mental Model Before Writing Any Code
2. Project Architecture — The Layered Thinking
3. Phase 1 — The Package: `rtl/i2c_pkg.sv`
4. Phase 1 — The Interface: `tb/i2c_if.sv`
5. Phase 2 — I2C Master RTL: `rtl/i2c_master_controller.sv`
6. Phase 2 — Master Wrapper: `rtl/i2c_master.sv`
7. Phase 3 — I2C Slave RTL: `rtl/i2c_slave_controller.sv`
8. Phase 3 — Slave Wrapper: `rtl/i2c_slave.sv`
9. Phase 4 — Top Testbench: `tb/top_tb.sv`
10. Phase 4 — UVM Sequence Item: `uvm/i2c_seq_item.sv`
11. Phase 4 — UVM Driver: `uvm/i2c_master_driver.sv`
12. Phase 4 — UVM Monitor: `uvm/i2c_monitor.sv`
13. Phase 4 — UVM Scoreboard: `uvm/i2c_scoreboard.sv`
14. Phase 6 — SVA Assertions: `sva/i2c_protocol_assertions.sv`
15. The Complete Bug Fix Log — What Went Wrong and Why
16. How Everything Connects — Full Data Flow
17. Interview Preparation — Questions This Project Answers

---

1. Mental Model Before Writing Any Code

1.1 Understanding I2C at the Physical Level

Before writing a single character of SystemVerilog, you must understand the protocol so deeply that you can draw the waveform from memory. If you cannot draw it, you cannot implement it correctly.

I2C uses exactly two wires:

- `SDA` — Serial Data Line. Bidirectional. Both master and slave can drive this.
- `SCL` — Serial Clock Line. Normally driven by master only (except clock stretching).

The most important concept: Open-Drain / Wired-AND

Both SDA and SCL are **open-drain**. This means:

- Any device on the bus can **pull the line LOW** (by connecting it to ground through a transistor).
- No device can actively drive the line HIGH. HIGH happens only when **everyone releases** the line and the external pull-up resistor pulls it up.

```
VCC
 |
[R]  ← Pull-up resistor (typically 4.7kΩ for standard mode)
 |
———————————————— SDA bus
 |            |
[NMOS]     [NMOS]
 |            |
GND          GND
Master       Slave

If Master pulls: SDA = 0
If Slave pulls:  SDA = 0
If neither pulls: SDA = 1 (pull-up wins)
If both pull:     SDA = 0 (wired-AND)
```

Why does this matter for your RTL?

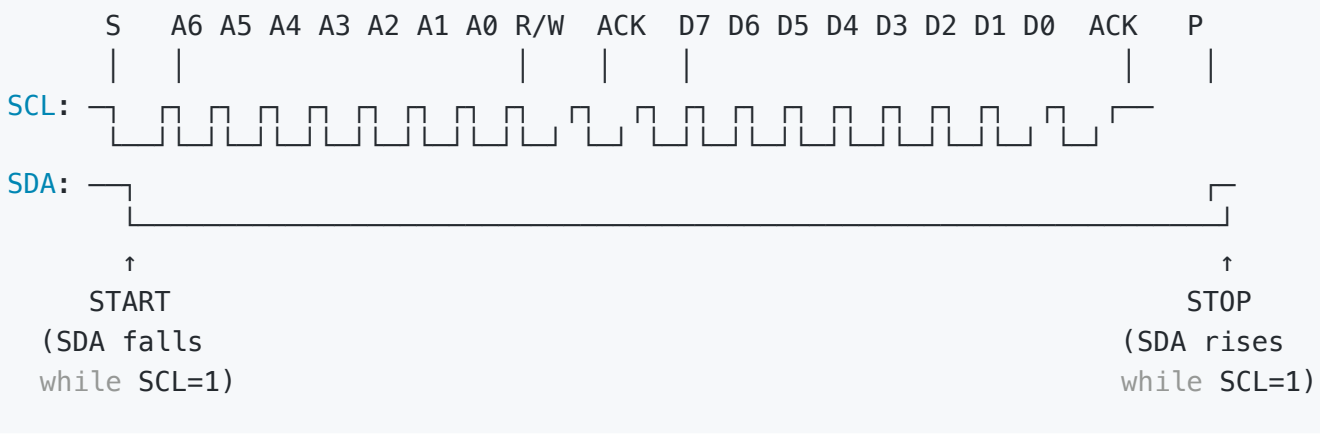
In your RTL, you cannot write `sda = 1'b1` to drive the line high. You must write:

```
assign sda = (sda_o == 1'b0) ? 1'b0 : 1'bz;
//                              ↑              ↑
//                         Active pull-low    Release (let pull-up win)
```

This is the **tri-state model**. The signal is either driven LOW (`1'b0`) or high-impedance (`1'bz` , meaning "I'm not driving anything").

1.2 Understanding the I2C Frame Format

A complete I2C write transaction looks like this:



START condition: SDA transitions from HIGH to LOW while SCL is already HIGH.
STOP condition: SDA transitions from LOW to HIGH while SCL is already HIGH.
Data bit: SDA must be STABLE while SCL is HIGH. Data can only change when SCL is LOW.
ACK: After 8 data bits, the RECEIVER pulls SDA LOW for one SCL cycle (the 9th clock).
NACK: If the receiver does not pull SDA LOW, it's a NACK.

1.3 The Key Questions to Answer Before Coding

Before writing any RTL, answer these explicitly:

1. **Who generates SCL?** → Master only (slave only touches it for clock stretching).
2. **Who drives SDA during address phase?** → Master.
3. **Who drives SDA during ACK after address?** → Slave (if address matched).
4. **Who drives SDA during data write?** → Master (it's transmitting data).
5. **Who drives SDA during ACK after data write?** → Slave.
6. **Who drives SDA during data read?** → Slave (it's transmitting data).
7. **Who drives SDA during ACK after data read?** → Master.
8. **What does "clock stretching" mean?** → Slave holds SCL LOW after master releases it, telling master to wait.

If you can answer all 8 correctly, you can implement I2C. If any answer is wrong, your bus will be garbage.

---

2. Project Architecture — The Layered Thinking

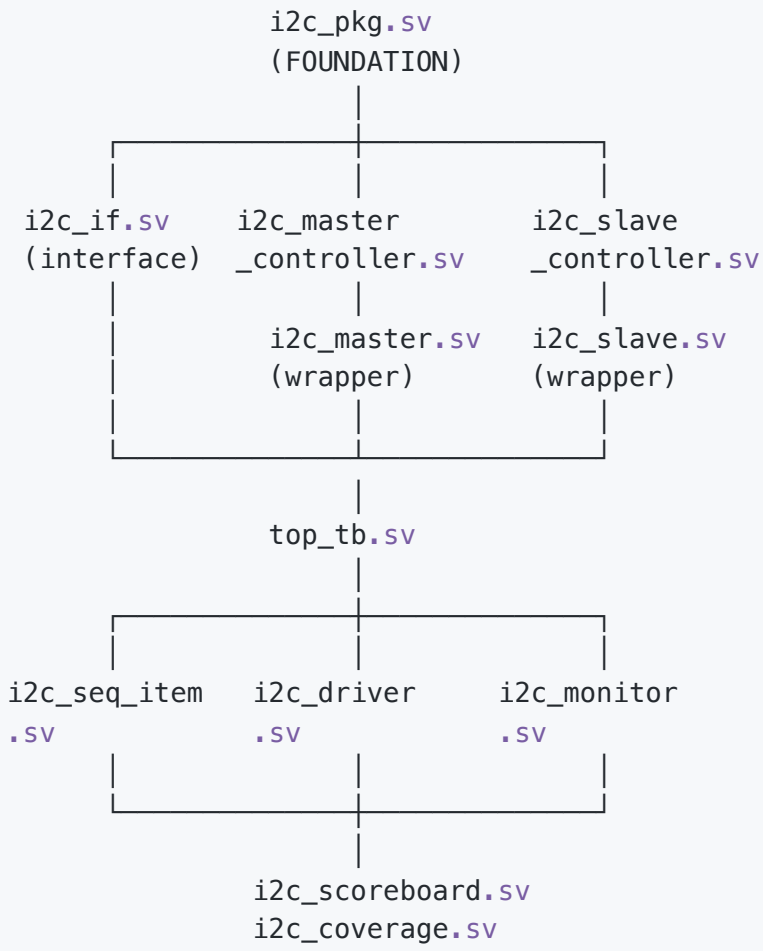
2.1 Why Layers Matter

The biggest mistake beginners make is trying to write everything at once. A professional engineer decomposes the problem into layers where:

- Each layer has exactly one responsibility.
- Each layer only depends on layers below it.
- Each layer can be tested independently.

```
┌──────────────────────────────────────────┐
│  LAYER 4: Tests, Assertions, Coverage    │  ← What we verify
├──────────────────────────────────────────┤
│  LAYER 3: UVM TB (Driver, Monitor, SB)   │  ← How we verify
├──────────────────────────────────────────┤
│  LAYER 2: RTL (Master + Slave)           │  ← What we're designing
├──────────────────────────────────────────┤
│  LAYER 1: Interface (Physical Bus)       │  ← How things connect
├──────────────────────────────────────────┤
│  LAYER 0: Package (Shared Types)         │  ← What everything uses
└──────────────────────────────────────────┘
```

2.2 File Dependency Graph



```
                    i2c_env.sv
                        |
                    i2c_tests.sv
                        |
                i2c_protocol_assertions.sv
```

Every file at the bottom depends on files above it. The package is at the very top because everything needs it.

---

3. Phase 1 — The Package: `rtl/i2c_pkg.sv`

3.1 Why a Package Exists

In a multi-file SystemVerilog project, you have a choice: put your constants, types, and enums directly in every file that needs them (duplicated, hard to maintain), or put them in ONE package that everyone imports.

A **package** in SystemVerilog is exactly like:

- A Python module (`import i2c_pkg`)
- A C++ namespace (`using namespace i2c_pkg`)
- A Java package

Everything inside a package becomes a shared namespace accessible anywhere with `import i2c_pkg::*` .

3.2 The Include Guard — Line 1 and 2

```
`ifndef I2C_PKG_SV    // Line 1
`define I2C_PKG_SV    // Line 2
```

What it does: Prevents the package from being compiled twice.

Why it matters: In a large project, multiple files include the same package. Without this guard, if two files include `i2c_pkg.sv` , the compiler sees the package declaration twice and throws a "redefinition" error. The guard says: "If I've already been compiled (`I2C_PKG_SV` is defined), skip me. If not, compile me and define the macro so I'm skipped next time."

Every `.sv` file in this project starts with an include guard. This is not optional — it's mandatory practice.

3.3 Timing Parameters — The Math That Drives Everything

```
parameter int CLK_FREQ_MHZ  = 50;    // System clock: 50 MHz
parameter int T_SCL_PERIOD  = 500;   // SCL period in system clock cycles
parameter int T_SCL_HIGH    = 250;   // SCL high time: 250 cycles
parameter int T_SCL_LOW     = 250;   // SCL low time: 250 cycles
```

The calculation:

```
I2C Standard Mode target: 100 kHz SCL
System clock: 50 MHz → period = 20 ns per clock cycle
SCL period = 1 / 100,000 Hz = 10,000 ns = 10 µs
Number of system clock cycles per SCL period = 10,000 ns / 20 ns = 500 cycles
```

So `T_SCL_PERIOD = 500` means "count 500 system clocks to make one SCL cycle."

`T_SCL_HIGH = 250` and `T_SCL_LOW = 250` give a 50% duty cycle. The master will:

- Drive SCL LOW for 250 clock cycles (5 µs)
- Drive SCL HIGH for 250 clock cycles (5 µs)
- Repeat

```
parameter int T_SU_STA = 10;   // Setup time for START condition
parameter int T_HD_STA = 10;   // Hold time after START
parameter int T_SU_STO = 10;   // Setup time before STOP
parameter int T_HD_DAT = 5;    // Data hold time after SCL falls
parameter int T_SU_DAT = 5;    // Data setup time before SCL rises
parameter int T_BUF    = 20;   // Bus free time between STOP and next START
```

These timing parameters come from the NXP UM10204 I2C specification tables. In Standard Mode (100 kHz), the spec requires:

- `t_SU;STA` (setup time for START) ≥ 4.7 µs → 10 cycles × 20 ns = 200 ns. We exceed the minimum, which is safe.
- `t_HD;STA` (hold time for START) ≥ 4.0 µs → 10 cycles = 200 ns. Safe.

In a real chip these would be tighter and more precisely calculated. For a verification project, being conservative (longer than minimum) is fine.

```
parameter int MAX_CLK_STRETCH = 1000;  // Max wait for slave to release SCL
```

This is the **clock stretch timeout**. If a slave holds SCL LOW for more than 1000 clock cycles (20 µs), the master declares a timeout. In the real world, a slave might hold SCL indefinitely (broken

slave). This bounds the wait.

3.4 The Command Enum — Why Not Just Use Logic[3:0]

```
typedef enum logic [3:0] {
    CMD_NOP       = 4'h0,   // No operation
    CMD_START     = 4'h1,   // Generate START condition
    CMD_STOP      = 4'h2,   // Generate STOP condition
    CMD_WRITE     = 4'h3,   // Write a byte (8 bits + ACK)
    CMD_READ_ACK  = 4'h4,   // Read a byte and send ACK
    CMD_READ_NACK = 4'h5,   // Read a byte and send NACK (last byte)
    CMD_REP_START = 4'h6,   // Repeated START
    CMD_ABORT     = 4'h7    // Abort and release bus
} i2c_cmd_e;
```

Why an enum instead of `logic [3:0]`?

With `logic [3:0]`, a bug might write `4'hF` as a command. Nothing catches it — it's just an unknown command. With an enum:

1. The simulator knows valid values. Writing `4'hF` is `i2c_cmd_e'(4'hF)` and simulators warn about undefined enum values.
2. Waveforms show "CMD_WRITE" instead of "4'h3" — vastly easier to debug.
3. Synthesis tools can optimize encoding.
4. Code is self-documenting.

The **`typedef`** keyword means "create a new type called `i2c_cmd_e`." The `_e` suffix is a naming convention meaning "enum." Following consistent naming conventions is how you write code that other engineers can read.

3.5 Direction and Status Enums

```
typedef enum logic {
    DIR_WRITE = 1'b0,   // R/W bit = 0 means master is writing
    DIR_READ  = 1'b1    // R/W bit = 1 means master is reading
} i2c_dir_e;
```

Critical insight: The last bit of the I2C address byte IS the R/W direction bit. By making `DIR_WRITE = 0` and `DIR_READ = 1`, you can construct the full address byte as:

```
addr_byte = {7_bit_address, direction};
// Example: address 0x50, write:
// addr_byte = {7'h50, DIR_WRITE} = {7'b101_0000, 1'b0} = 8'b1010_0000 = 0xA0
```

```
// Example: address 0x50, read:
// addr_byte = {7'h50, DIR_READ}  = {7'b101_0000, 1'b1} = 8'b1010_0001 = 0xA1
```

This is elegant because the direction bit is literally concatenated. No bit manipulation needed.

```
typedef enum logic [2:0] {
    STS_IDLE     = 3'b000,  // Ready for commands
    STS_BUSY     = 3'b001,  // Transaction in progress
    STS_ACK      = 3'b010,  // ACK received/sent
    STS_NACK     = 3'b011,  // NACK received/sent
    STS_ARB_LOST = 3'b100,  // Bus arbitration lost (multi-master)
    STS_TIMEOUT  = 3'b101,  // Clock stretch exceeded limit
    STS_DONE     = 3'b110,  // Transaction completed successfully
    STS_ERR      = 3'b111   // Generic error
} i2c_status_e;
```

The status enum is what the master reports back to the CPU after each operation. The CPU polls this register to know if the I2C transaction is done, succeeded, or failed.

3.6 The Register Map Enum

```
typedef enum logic [2:0] {
    REG_CTRL   = 3'h0,   // Control: enable, mode selection
    REG_CMD    = 3'h1,   // Write here to issue a command
    REG_STATUS = 3'h2,   // Read here to check current state
    REG_DATA   = 3'h3,   // Write: data to transmit. Read: data received
    REG_ADDR   = 3'h4,   // Slave address for auto-addressing
    REG_IRQ_EN = 3'h5,   // Interrupt enable mask
    REG_IRQ_ST = 3'h6    // Interrupt status (read-to-clear)
} i2c_reg_e;
```

This defines the APB-like register interface between a CPU and the I2C master. The CPU writes to a 3-bit address to access different registers. This is how real hardware works — ARM Cortex-M processors communicate with peripherals through memory-mapped register interfaces just like this.

3.7 The Helper Function

```
function automatic addr_byte_t build_addr_byte(
    logic [6:0] addr,
    i2c_dir_e   dir
);
    return {addr, dir};   // Concatenate 7-bit addr with 1-bit direction
endfunction
```

`automatic` means this function has automatic variable lifetime (each call gets its own stack frame, like C functions). Without `automatic`, functions in SystemVerilog share static storage and concurrent calls corrupt each other.

`addr_byte_t` is defined as `typedef logic [7:0] addr_byte_t` — just a convenient alias for an 8-bit value that specifically represents an I2C address byte.

---

4. Phase 1 — The Interface: `tb/i2c_if.sv`

4.1 What a SystemVerilog Interface Is

Without interfaces, your testbench has:

```
// Tedious, error-prone, non-reusable
module top_tb;
  wire sda, scl;
  i2c_master dut_m (.sda(sda), .scl(scl), ...30 more ports...);
  i2c_slave  dut_s (.sda(sda), .scl(scl), ...20 more ports...);
  // Driver code: need to drive sda, scl by name as separate signals
endmodule
```

With an interface:

```
// Clean, bundled, reusable
interface i2c_if(input logic clk, input logic rst_n);
  wire sda;
  wire scl;
  // All tasks for driving the bus live here
endinterface

module top_tb;
  i2c_if i2c_bus_if(clk, rst_n);
  i2c_master dut_m (.bus(i2c_bus_if.master));  // Entire interface in one port
endmodule
```

The interface acts as a **bundle** — all related signals travel together, and tasks for driving/monitoring the bus are co-located with the signals they operate on.

4.2 Why `wire` and Not `logic`

```
wire sda;   // NOT logic sda
wire scl;   // NOT logic scl
```

This is critical. `logic` in SystemVerilog can have **exactly one driver**. If you use `logic sda` and both the master driver and slave driver try to drive it, you get a multi-driver error.

`wire` supports **multiple drivers** and resolves the driven values:

```
Master drives 0, Slave drives z → wire resolves to 0
Master drives z, Slave drives 0 → wire resolves to 0
Master drives z, Slave drives z → wire resolves to z (pull-up gives 1)
Master drives 0, Slave drives 0 → wire resolves to 0
Master drives 1, Slave drives 0 → wire resolves to X (conflict! both actively driving)
```

The last case (X) would be a bug — in real open-drain hardware, two devices driving different values would fight, potentially damaging the transistors. In simulation, X correctly signals "something is wrong."

4.3 The Modports

```
modport master  (inout sda, inout scl, input clk, input rst_n);
modport slave   (inout sda, inout scl, input clk, input rst_n);
modport monitor (input  sda, input  scl, input clk, input rst_n);
```

Modports define **roles**. Each role specifies which signals a module can drive and which it can only read.

- `master` : Can drive SDA and SCL (it's the transmitter of clock and data).
- `slave` : Can also drive SDA and SCL (for ACK and clock stretching).
- `monitor` : Can only read SDA and SCL (it's passive — never drives the bus).

If a module connected via the `monitor` modport tries to write to `sda` , the compiler catches it at elaboration time. This is compile-time enforcement of design intent.

4.4 Tasks for Bus Driving

Tasks inside the interface are extremely useful because they co-locate bus-driving behavior with the signals. The UVM driver calls these tasks.

```
task automatic start_cond();
    sda = 1'b1;    // Step 1: Ensure SDA is HIGH
    scl = 1'b1;    // Step 2: Ensure SCL is HIGH (bus must be idle)
    #50;           // Step 3: Wait for setup time (in TB, #delays are OK)
    sda = 1'b0;    // Step 4: SDA FALLS while SCL is still HIGH → START
    #50;           // Step 5: Hold time
    scl = 1'b0;    // Step 6: SCL falls (data phase begins)
endtask
```

Why is the sequence `SDA↓ then SCL↓` and not `SCL↓ then SDA↓`?

The I2C spec defines START as "SDA falls while SCL is HIGH." If SCL went low first, then SDA falling would just be a normal data change during a low SCL phase — not a START condition. The order is protocol-mandated.

```
task automatic send_byte(
    input  logic [7:0] data,
    output logic       ack
);
    for (int i = 7; i >= 0; i--) begin  // i = 7 first → MSB first
        scl = 1'b0;           // SCL low: safe to change SDA
        sda = data[i];        // Drive the current bit
        #50;                  // Data setup time
        scl = 1'b1;           // SCL high: slave samples SDA
        #50;                  // Data hold time
        scl = 1'b0;           // SCL low: ready for next bit
        #50;
    end
    // 9th clock cycle: ACK/NACK
    sda = 1'bz;               // Release SDA — let slave drive it
    #50;
    scl = 1'b1;               // SCL high: sample the 9th bit
    #10;
    ack = sda;                // ack=0 means slave pulled SDA low (ACK)
    #40;                       // ack=1 means slave released (NACK)
    scl = 1'b0;
    sda = 1'bz;
endtask
```

The loop `for (int i = 7; i >= 0; i--)`:

- Starts at bit 7 (MSB) and counts down to bit 0 (LSB).
- `data[7]` is the most significant bit.
- I2C transmits MSB first — this is a protocol requirement.

Why release SDA (`1'bz`) for the ACK bit?

- The master has finished transmitting. The 9th clock cycle belongs to the RECEIVER.
- If the receiver (slave) wants to ACK, it pulls SDA LOW.
- If the master held SDA LOW, it would force an ACK even if the slave wanted to NACK — wrong.
- By releasing to `1'bz`, the master lets whoever wants to drive SDA do so.

---

5. Phase 2 — I2C Master RTL:

`rtl/i2c_master_controller.sv`

5.1 Architecture Decision: Byte-Level FSM

Before writing any state machine code, you choose the **abstraction level** of your FSM.

Option A: Bit-Level FSM

```
One state per bit: TX_BIT_7, TX_BIT_6, TX_BIT_5, ..., TX_BIT_0
Pros: Simple per-state logic
Cons: 8 states just for one byte direction, 16 for TX+RX, doesn't scale
```

Option B: Byte-Level FSM with Counter

```
One state for all 8 bits: TX_BYTE
Internal bit counter tracks which bit we're on
Pros: Fewer states, cleaner, mirrors how protocols actually work
Cons: Slightly more complex per-state logic (need counter)
```

I chose Option B (byte-level). This is the industry standard approach. The bit counter is a simple 3-bit register that increments each time a bit period completes.

5.2 The Open-Drain Output Model

```
logic sda_o, scl_o;    // Internal "what we want to drive"
logic sda_i, scl_i;    // Internal "what's on the bus"

assign sda = (sda_o == 1'b0) ? 1'b0 : 1'bz;   // Tri-state SDA
assign scl = (scl_o == 1'b0) ? 1'b0 : 1'bz;   // Tri-state SCL
assign sda_i = sda;    // Tap the actual bus value for sampling
assign scl_i = scl;    // Tap SCL to detect clock stretching
```

Why this pattern?

`sda_o` is the "intent" — what the master WANTS to drive. But in open-drain hardware, you cannot force HIGH. So:

- If `sda_o = 0`: Drive SDA to `1'b0` (pull low)
- If `sda_o = 1`: Drive SDA to `1'bz` (release — let pull-up or slave determine value)

The `assign sda_i = sda` line is critical for two things:

1. **ACK detection**: After sending a byte, the master releases SDA. Then reads `sda_i` to see if slave pulled it low (ACK).
2. **Clock stretch detection**: After master releases SCL (sets `scl_o = 1`), it reads `scl_i`. If the slave is stretching, `scl_i` will still be 0 even though `scl_o = 1`.

5.3 The State Machine Declaration

```
typedef enum logic [4:0] {
    STATE_IDLE,           // Waiting for host command
    STATE_START_1,        // SDA falling for START
    STATE_START_2,        // SCL falling after START
    STATE_TX_BIT,         // Transmitting 8 data bits (with internal bit counter)
    STATE_RX_BIT,         // Receiving 8 data bits
    STATE_ACK_WAIT,       // 9th clock — waiting for slave ACK
    STATE_ACK_SEND,       // 9th clock — master drives ACK or NACK to slave
    STATE_STOP_1,         // SDA low, SCL rising
    STATE_STOP_2,         // SDA rising while SCL high → STOP
    STATE_REP_START_1,    // Repeated START phase 1
    STATE_REP_START_2     // Repeated START phase 2
} state_e;

state_e state, next_state;
```

`state` is the current registered state. `next_state` is the combinational next-state logic output. This is the standard two-always-block FSM pattern:

- One `always_ff` for the state register (sequential).
- One `always_comb` for the next-state and output logic (combinational).

Why `logic [4:0]` ? The enum needs enough bits to represent all values. 11 states fit in 4 bits (max 16 values), but `logic [4:0]` is used to be safe and allow future state additions.

5.4 The Timer — The Heartbeat of the Master

The timer controls exactly how long each phase (SCL low, SCL high, setup times) lasts.

```
logic [$clog2(T_SCL_PERIOD)-1:0] timer;
```

`$clog2(T_SCL_PERIOD)` calculates the ceiling of $\log_2(500)$ = ceiling(8.96) = 9 bits. So the timer is 9 bits wide, enough to count from 0 to 511 (which covers T_SCL_PERIOD = 500).

The timer logic (fixed version):

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
```

```
        timer         <= '0;
        timer_expired <= 1'b0;
    end else if (timer_clr) begin         // Reset timer on state change
        timer         <= '0;
        timer_expired <= 1'b0;
    end else begin
        if (timer == T_SCL_PERIOD - 1) begin
            timer         <= '0;          // Wrap around
            timer_expired <= 1'b1;        // Signal that period is done
        end else begin
            timer         <= timer + 1'b1;
            timer_expired <= 1'b0;
        end
    end
end

assign timer_clr = (state != next_state);  // Clear when changing states
```

Why `timer_clr = (state != next_state)`?

When the FSM transitions from one state to another (e.g., `STATE_START_1` → `STATE_START_2`), the timer must reset to 0 so the new state gets a fresh full period. If we don't reset, a state might only get a partial period because the timer was already partway through.

Why `timer_expired` as a registered signal rather than just `(timer == T_SCL_PERIOD - 1)`?

The combinational version is true for exactly one clock cycle (when timer equals the max). Making it registered ensures it's stable at the clock edge when the FSM reads it. This avoids combinational glitches propagating into the FSM.

5.5 SCL Generation — The Fixed Logic

This is where the first critical bug lived.

The broken version:

```
STATE_TX_BIT: scl_o <= (timer < T_SCL_HIGH) ? 1'b0 : 1'b0;
//                                                    ↑
//                              BOTH BRANCHES RETURN 0! SCL NEVER GOES HIGH!
```

Why this happened: Copy-paste error during writing. The intent was "LOW for first half, HIGH for second half." But both values were 0.

The fixed version:

```
STATE_TX_BIT: begin
    if      (timer < T_SCL_LOW)                    scl_o <= 1'b0;  // First 250 cycles:
```

```
    else if (timer < T_SCL_LOW + T_SCL_HIGH)      scl_o <= 1'b1;  // Next 250 cycles:
    else                                           scl_o <= 1'b0;  // Reset phase
end
```

What this produces on the bus:

```
timer:  0    1    2  ... 249  250  251 ... 499  500(reset to 0)
SCL:    0    0    0  ...  0    1    1  ...  1       0
        └──────────────────┘    └──────────────────┘
             LOW (250 cycles)      HIGH (250 cycles)
             = 5 µs                = 5 µs
```

The same pattern applies to other states:

```
STATE_ACK_WAIT: scl_o <= (timer < T_SCL_LOW) ? 1'b0 : 1'b1;
// 9th clock cycle uses the same timing as data bits
```

5.6 SDA Output Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sda_o <= 1'b1;    // On reset: release SDA (bus idle = HIGH)
    end else begin
        case (state)
            STATE_IDLE:        sda_o <= 1'b1;          // Bus idle
            STATE_START_1:     sda_o <= 1'b0;          // SDA falls for START
            STATE_START_2:     sda_o <= 1'b0;          // Keep low
            STATE_TX_BIT:      sda_o <= shift_reg[7];  // MSB first
            STATE_ACK_WAIT:    sda_o <= 1'b1;          // RELEASE for slave to ACK
            STATE_ACK_SEND:    sda_o <= 1'b0;          // Drive ACK (master→slave)
            STATE_STOP_1:      sda_o <= 1'b0;          // SDA low before STOP
            STATE_STOP_2:      sda_o <= 1'b1;          // SDA rises = STOP
            STATE_REP_START_1: sda_o <= 1'b1;          // SDA goes high first
            STATE_REP_START_2: sda_o <= 1'b0;          // Then falls = Sr
            default:           sda_o <= 1'b1;
        endcase
    end
end
```

Every state and why:

- `STATE_IDLE` : Bus must be HIGH when idle (both SDA and SCL). Any activity starting from this state needs SDA to already be HIGH for START to be valid.
- `STATE_START_1` : This IS the START condition. SDA falls while SCL is still HIGH (from STATE_IDLE). First clock of falling SDA.

- `STATE_TX_BIT` : `shift_reg[7]` is the current bit to send. The shift register is loaded before entering this state. It shifts left on each bit period so the next MSB is ready.
- `STATE_ACK_WAIT` : CRITICAL. The master must release SDA here so the slave can pull it LOW to ACK. If master kept SDA LOW, the slave could never NACK.
- `STATE_STOP_1` : SDA must be LOW before the STOP, because STOP is defined as SDA RISING while SCL is HIGH. You can't rise if you start HIGH.
- `STATE_STOP_2` : SDA rises while SCL is HIGH → this IS the STOP condition.

5.7 The Shift Register

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        shift_reg <= 8'h00;
    end else begin
        if (cpu_wr && cpu_addr == REG_DATA) begin
            shift_reg <= cpu_wdata;                      // Load: CPU writes data to
        end else if (state == STATE_RX_BIT && scl_rise) begin
            shift_reg <= {shift_reg[6:0], sda_i};        // Receive: shift in from bus
        end else if (state == STATE_TX_BIT && timer_expired) begin
            shift_reg <= {shift_reg[6:0], 1'b0};         // Transmit: shift out (MSB l
        end
    end
end
```

TX path — how shifting works:

Starting with data `0xA5 = 1010_0101` :

```
Cycle 1: shift_reg = 1010_0101, sda_o = shift_reg[7] = 1 (sending bit 7)
         After timer_expired: shift_reg = {0100_1010, 0} = 0100_1010
Cycle 2: shift_reg = 0100_1010, sda_o = shift_reg[7] = 0 (sending bit 6)
         After timer_expired: shift_reg = {100_1010_0} = 1001_0100
Cycle 3: shift_reg = 1001_0100, sda_o = shift_reg[7] = 1 (sending bit 5)
...
Cycle 8: bit 0 is sent
```

The left-shift `{shift_reg[6:0], 1'b0}` moves bit 6 to position 7, bit 5 to position 6, etc. The LSB gets a 0 padding (doesn't matter since it's already been shifted out).

RX path:

```
shift_reg <= {shift_reg[6:0], sda_i};  // Shift in from right, MSB fills from left
```

Opposite direction: new bit arrives at LSB position, existing bits shift left.

```
Cycle 1: sda_i = 1 (bit 7 from slave), shift_reg = 0000_0001
Cycle 2: sda_i = 0 (bit 6),           shift_reg = 0000_0010
...
Cycle 8: shift_reg = 1010_0101 (complete byte received)
```

5.8 The ACK Registration Fix

The broken version:

```
assign ack_received = (state == STATE_ACK_WAIT) && (sda_i == 1'b0) && scl_i;
```

Why this is wrong: This is combinational logic. `sda_i` is a live bus signal. During the 9th clock cycle:

- SCL goes high, stays high for T_SCL_HIGH cycles, then goes low.
- SDA might be noisy during transitions.
- The combinational `ack_received` could glitch as SDA transitions.
- If `ack_received` is used in the next-state logic (also combinational), a glitch could cause a spurious state transition.

The fixed version:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ack_received <= 1'b0;
    end else if (state == STATE_ACK_WAIT && scl_rise) begin
        ack_received <= (sda_i == 1'b0);  // Sample ONCE on SCL rising edge
    end else if (state != STATE_ACK_WAIT) begin
        ack_received <= 1'b0;             // Clear when not in ACK state
    end
end
```

Why sample on `scl_rise`?

The I2C spec says: "SDA must be stable when SCL is HIGH." The legally correct sampling point is while SCL is HIGH. The rising edge of SCL marks the beginning of the SCL high period — the exact moment when data is guaranteed valid. Sampling here gives a clean, stable value.

5.9 The Multi-Driver Bug Fix

The broken version:

```
// In always_comb:
always_comb begin
    next_state = state;
    // ... case logic ...
end

// In always_ff:
always_ff @(posedge clk) begin
    state <= next_state;
    // ...
    if (timeout) next_state <= STATE_IDLE;  // ← MULTI-DRIVER ON next_state!
end
```

Why this is wrong: `next_state` is driven by the `always_comb` block. Driving it again from `always_ff` creates **two drivers on the same net**. This is a hardware error — you cannot have two flip-flops driving the same wire in hardware without a conflict. The simulator gives an X or multi-driver warning.

The fixed version:

```
always_comb begin
    next_state = state;
    if (timeout) begin
        next_state = STATE_IDLE;    // ← Timeout override ONLY in comb block
    end else begin
        case (state)
            // ... all state transitions ...
        endcase
    end
end
```

The timeout escape is part of the combinational next-state logic. There is now exactly ONE driver of `next_state`.

5.10 The Next-State Logic

```
STATE_TX_BIT: begin
    if (timer_expired) begin
        if (bit_cnt == 3'd7) begin
            next_state = STATE_ACK_WAIT;   // All 8 bits sent → wait for ACK
        end else begin
            next_state = STATE_TX_BIT;     // More bits remain → stay
        end
    end else begin
        next_state = STATE_TX_BIT;         // Bit period not done → stay
```

```
    end
end
```

The bit counter:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        bit_cnt <= 3'd0;
    end else begin
        if (next_state != state) begin
            bit_cnt <= 3'd0;              // Reset on any state change
        end else if ((state == STATE_TX_BIT || state == STATE_RX_BIT) && timer_expired
            bit_cnt <= bit_cnt + 1'b1;    // Increment after each bit period
        end
    end
end
```

`bit_cnt` counts 0 to 7 (8 bits). When it reaches 7 and the timer expires, the 8th bit has been sent. The FSM transitions to `STATE_ACK_WAIT`.

---

6. Phase 2 — Master Wrapper: `rtl/i2c_master.sv`

6.1 Why a Wrapper Exists

The wrapper pattern separates **functionality** (the controller) from **integration** (the top-level connections). The controller module has all the logic. The wrapper just connects it to the outside world.

Benefits:

- You can swap the controller without changing the top-level port list.
- You can add multiple controllers (e.g., a second I2C master) with the same wrapper pattern.
- Tests instantiate the wrapper (realistic) or the controller directly (isolated).

6.2 The Broken Wrapper (Before Fix)

The AI-generated wrapper had a catastrophic bug:

```
module i2c_master (
    input logic [2:0] cpu_addr,
    ...
```

```
    // INSIDE THE PORT LIST:
    cpu_interface cpu_if(.clk, .rst_n, ...);  // ← Interface instantiation inside port
    logic [2:0] cpu_addr;  // ← Re-declaring a port that's already declared!
```

Why is this wrong? Module port lists declare the module's external interface. You cannot instantiate sub-modules inside a port list — that goes in the module body. The duplicate port declaration causes a "redefinition" error. This file would not even parse.

6.3 The Fixed Wrapper

```
module i2c_master (
    input  logic        clk,
    input  logic        rst_n,
    input  logic [2:0]  cpu_addr,
    input  logic        cpu_wr,
    input  logic [7:0]  cpu_wdata,
    output logic [7:0]  cpu_rdata,
    output logic        cpu_irq,
    inout  wire         scl,
    inout  wire         sda,
    output logic        busy,
    output logic        done,
    output logic        ack,
    output logic        nack,
    output logic        timeout,
    output logic        bus_error
);
    i2c_master_controller master_core (
        .clk      (clk),       // Connect every port by name (not position)
        .rst_n    (rst_n),
        .cpu_addr (cpu_addr),
        .cpu_wr   (cpu_wr),
        .cpu_wdata(cpu_wdata),
        .cpu_rdata(cpu_rdata),
        .cpu_irq  (cpu_irq),
        .scl      (scl),
        .sda      (sda),
        .busy     (busy),
        .done     (done),
        .ack      (ack),
        .nack     (nack),
        .timeout  (timeout),
        .bus_error(bus_error)
    );
endmodule
```

Why named port connections (`.port(signal)`) instead of positional (`signal1, signal2, ...`)?

Positional connections are order-dependent. If you change the port order in the controller, the wrapper silently connects wrong signals. Named connections are order-independent and self-documenting. Always use named connections in professional code.

---

7. Phase 3 — I2C Slave RTL: `rtl/i2c_slave_controller.sv`

7.1 The Slave's Mindset

The slave is fundamentally different from the master:

- **Reactive, not proactive**: Slave does nothing until it sees a START condition.
- **No clock generation**: The slave samples SCL, doesn't generate it.
- **Address-aware**: Slave only responds if the address byte matches its own address.
- **Register bank**: Slave has 8 internal registers that store written data and serve read data.

7.2 START and STOP Detection

This is the slave's most important logic:

```
logic sda_d;     // Delayed SDA (previous cycle value)
logic start_cond;
logic stop_cond;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) sda_d <= 1'b1;
    else        sda_d <= sda_i;    // Register SDA with 1-cycle delay
end

// START: SDA transitions LOW→HIGH... wait, no.
// START: SDA transitions HIGH→LOW while SCL is HIGH
assign start_cond = (sda_i == 1'b0 &&    // Current SDA is LOW
                     sda_d == 1'b1 &&    // Previous SDA was HIGH (falling edge)
                     scl_i == 1'b1);     // SCL is HIGH right now

// STOP: SDA transitions LOW→HIGH while SCL is HIGH
assign stop_cond  = (sda_i == 1'b1 &&    // Current SDA is HIGH
                     sda_d == 1'b0 &&    // Previous SDA was LOW (rising edge)
                     scl_i == 1'b1);     // SCL is HIGH right now
```

The edge detection pattern:

- `sda_d` is SDA from the previous clock cycle.

- Comparing `sda_i` (current) with `sda_d` (previous) detects edges.
- Adding `scl_i == 1'b1` gates the detection to only when SCL is high.

Why not use `@(negedge sda)` directly?

In synthesizable RTL, you only use one clock edge (`@(posedge clk)`). Using `@(negedge sda)` would mean the logic clocked on SDA — not synthesizable as digital logic. The delayed-comparison pattern achieves the same result with standard synchronous logic.

7.3 The Slave FSM

```
typedef enum logic [3:0] {
    SLV_IDLE,       // Waiting for START
    SLV_START,      // START detected, begin receiving
    SLV_ADDR,       // Shifting in 7-bit address + R/W bit
    SLV_ACK,        // Drive ACK if address matches
    SLV_RX,         // Receiving data bytes from master (write transaction)
    SLV_RX_ACK,     // Sending ACK to master after each received byte
    SLV_TX,         // Transmitting data bytes to master (read transaction)
    SLV_TX_ACK,     // Waiting for ACK/NACK from master after each sent byte
    SLV_STRETCH,    // Holding SCL low (clock stretching)
    SLV_STOP        // STOP detected, release bus
} slv_state_e;
```

The transition that catches most bugs:

```
SLV_ADDR → SLV_ACK: After receiving 8 bits (7-bit address + R/W bit)
SLV_ACK  → SLV_RX:  If R/W bit = 0 (master is writing to slave)
SLV_ACK  → SLV_TX:  If R/W bit = 1 (master is reading from slave)
```

7.4 The Register Bank Write Bug and Fix

The broken version:

```
if (state == SLV_RX_ACK && active && rx_rw == 1'b0) begin
    internal_regs[reg_ptr] <= {shift_reg[6:0], sda_i};  // ← BUG
end
```

The problem: At `SLV_RX_ACK`, the master has just finished sending a data byte and is now driving the ACK/NACK bit. At this moment, `sda_i` is the ACK/NACK bit (bit 9), NOT the 8th data bit (bit 8 from the master's perspective). By using `{shift_reg[6:0], sda_i}`, you're ORing the last 7 data bits with the ACK bit. The stored byte will be wrong.

Why does this happen?

The shift register works like this:

- In `SLV_RX`, each rising SCL edge: `shift_reg <= {shift_reg[6:0], sda_i}`
- After 8 edges (bits 7 down to 0), `shift_reg` should contain the complete byte.
- But the code samples `sda_i` ONE MORE TIME in `SLV_RX_ACK`, which is now the ACK bit.

The fix — latch on the last data bit, not the ACK bit:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        latched_rx_byte <= 8'h00;
    end else if (state == SLV_RX && bit_cnt == 3'd7 && scl_fall) begin
        // Latch WHEN: we're in receive state, all 8 bits have been received,
        // and SCL is FALLING (the 8th bit has been stable during the high period)
        latched_rx_byte <= {shift_reg[6:0], sda_i};
        // At THIS moment, sda_i IS the 8th data bit (bit 0 of the byte)
        // because bit_cnt == 7 means we're receiving the 8th clock
        // and scl_fall means the SCL high period just ended
    end
end

// Then write the latched byte in SLV_RX_ACK:
if (state == SLV_RX_ACK && active_int && rx_rw == 1'b0) begin
    internal_regs[reg_ptr] <= latched_rx_byte;   // ← Correct byte, not ACK bit
end
```

7.5 The TX Mode Fix

The broken version:

```
SLV_TX: begin
    if (active) sda_o <= 1'bz;   // ← WRONG: releases bus instead of driving data
end
```

Why this is wrong: In read mode, the slave is the transmitter. When the master does a read (R/W = 1), the slave must drive data bits onto SDA. Driving `1'bz` releases the bus — the master sees all 1s (HIGH, because pull-up). The data is lost.

The fix — a dedicated TX shift register:

```
logic [7:0] tx_shift_reg;

// Load the register data when entering TX mode:
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx_shift_reg <= 8'h00;
```

```
    end else if (state == SLV_ACK && addr_matched && rx_rw == 1'b1) begin
        // R/W=1 means master wants to read → load first register
        tx_shift_reg <= internal_regs[reg_ptr];
    end else if (state == SLV_TX && scl_fall) begin
        // After each bit sent (on SCL falling edge), shift left
        tx_shift_reg <= {tx_shift_reg[6:0], 1'b0};
    end
end

// Drive SDA with MSB of TX shift register:
SLV_TX: begin
    if (active_int) sda_o <= tx_shift_reg[7];   // MSB first
    else            sda_o <= 1'b1;              // Release if not active
end
```

The timing of loading: `tx_shift_reg` is loaded in `SLV_ACK` when `rx_rw == 1'b1`. Why here? Because `SLV_ACK` is the state right before `SLV_TX`. Loading in `SLV_ACK` means by the time the FSM enters `SLV_TX` and starts sending bit 7, `tx_shift_reg` already contains the correct register data.

---

8. Phase 3 — Slave Wrapper: `rtl/i2c_slave.sv`

The slave wrapper follows the same pattern as the master wrapper. One addition is the `SLAVE_ADDR` parameter:

```
module i2c_slave
#(
    parameter logic [6:0] SLAVE_ADDR = 7'h50   // Default address 0x50
)
```

Why parameterize the address?

If the address were hardcoded, every slave instance would have the same address. In a multi-slave testbench (test T11: multi-slave test), you instantiate multiple slaves:

```
i2c_slave #(.SLAVE_ADDR(7'h50)) slave_0 (...);
i2c_slave #(.SLAVE_ADDR(7'h51)) slave_1 (...);
i2c_slave #(.SLAVE_ADDR(7'h52)) slave_2 (...);
```

Each slave has a different address but the same implementation. Parameters enable reuse without code duplication.

---

9. Phase 4 — Top Testbench: `tb/top_tb.sv`

9.1 The Testbench's Role

The testbench is the simulation environment. It:

1. Generates clock and reset.
2. Instantiates the DUTs (master and slave).
3. Instantiates the interface (the bus).
4. Connects DUTs to the interface.
5. Starts the UVM test infrastructure.

The testbench itself has NO test logic — tests are in separate UVM test classes. This separation allows you to run different tests without modifying or recompiling the testbench.

9.2 Clock and Reset Generation

```
logic clk;
logic rst_n;

initial begin
    clk = 1'b0;
    forever #10 clk = ~clk;    // Toggle every 10 ns → 20 ns period → 50 MHz
end

initial begin
    rst_n = 1'b0;    // Assert reset
    #100;            // Hold reset for 100 ns (5 clock cycles)
    rst_n = 1'b1;    // Release reset
end
```

Why `forever #10 clk = ~clk`?

`#10` means "delay 10 ns." Toggling every 10 ns gives a period of 20 ns (10 ns HIGH + 10 ns LOW), which is 1/20ns = 50 MHz.

The `initial` block with `forever` runs until `$finish`. This is a standard clock generator pattern for simulation.

Why 100 ns reset? Five clock cycles. Registers need at least one clock edge after reset is released to initialize. Five cycles provides margin for any reset synchronization.

9.3 The UVM Kickoff

```
initial begin
    uvm_config_db #(virtual i2c_if)::set(null, "*", "i2c_vif", i2c_bus_if);
    run_test();
end
```

`uvm_config_db`: This is UVM's global configuration database. The testbench is storing a virtual interface handle (`i2c_bus_if`) in the database with the key `"i2c_vif"` . Any UVM component anywhere in the hierarchy can retrieve it with:

```
uvm_config_db #(virtual i2c_if)::get(this, "", "i2c_vif", vif);
```

This is how the testbench passes the interface to the driver and monitor without direct module hierarchical references. The driver just says "give me the interface named `i2c_vif` " and the database delivers it.

`run_test()`: Looks at the `+UVM_TESTNAME=<name>` simulator argument and instantiates the corresponding test class. This is what allows you to run different tests without recompiling:

```
xrun ... +UVM_TESTNAME=basic_write_test
xrun ... +UVM_TESTNAME=random_directed_test
```

9.4 The Floating Input Bug Fix

The broken version:

```
i2c_slave slave_dut (
    ...
    // stretch_en and stretch_cycles NOT connected → floating!
);
```

Why floating inputs are dangerous: In simulation, an unconnected input has the value `Z` (high impedance). When the logic inside the slave samples `stretch_en` , it sees `Z` . Boolean operations on `Z` produce `X` (unknown). Once `X` enters a register, it spreads everywhere (X-propagation). Your entire simulation becomes X.

The fix:

```
i2c_slave slave_dut (
    ...
    .stretch_en    (1'b0),      // Explicitly disabled
    .stretch_cycles(16'd0),     // Zero stretch cycles
);
```

Always connect every port. Use `1'b0` or `'0` for unused inputs. Use `()` only for unused outputs (not inputs).

---

10. Phase 4 — UVM Sequence Item: `uvm/i2c_seq_item.sv`

10.1 What a Sequence Item Is

A sequence item is a **transaction-level object** — the currency of UVM communication. Instead of thinking "I need to toggle SDA 72 times for a 9-byte transaction," you think "I want to send address 0x50, write 4 bytes."

The sequence item captures everything about one logical I2C transaction:

- Where to send it (address)
- What direction (read/write)
- What data
- How many bytes
- Any special conditions (repeated START, error injection)

10.2 The Factory Registration

```
`uvm_object_utils(i2c_seq_item)
```

This macro expands to code that registers `i2c_seq_item` with the UVM factory. The factory allows you to substitute one class for another at runtime. For example, to inject errors, you can create `i2c_error_seq_item extends i2c_seq_item` and tell the factory "whenever someone creates an i2c_seq_item, give them an i2c_error_seq_item instead." This is done without modifying any test code.

10.3 Randomizable Fields

```
rand logic [6:0]  addr;       // 7-bit slave address (randomized)
rand i2c_dir_e    dir;        // Read or write (randomized)
rand logic [7:0]  data [$];   // Queue of data bytes (randomized)
rand int          num_data;   // Number of data bytes (randomized, constrained)
rand bit          start_cond; // Should this transaction have a START?
rand bit          stop_cond;  // Should this transaction have a STOP?
rand bit          rep_start;  // Should this use Repeated START?
```

```
rand bit          inject_nack;    // Error injection
rand bit          inject_timeout; // Error injection
```

`rand` keyword: Marks a field as randomizable. When you call `item.randomize()`, the solver assigns random values to all `rand` fields, subject to constraints.

`data [$]`: A queue (dynamic array with push/pop). Length is not fixed at compile time. The constraint `data.size() == num_data` ensures the queue has exactly `num_data` elements after randomization.

10.4 Constraints — The Intelligence of Random Testing

Constraints define the legal space of random values. Without constraints, the solver might generate address `7'h00` (general call address) or data `8'bx` (unknown), causing false failures.

```
constraint valid_addr_c {
    addr != 7'h00;              // Not general call address
    addr != 7'h7F;              // Not reserved
    addr[6:3] != 4'b0000;       // Reserved range: 0000_XXX
    addr[6:3] != 4'b1111;       // Reserved range: 1111_XXX
}
```

These constraints come directly from the NXP UM10204 spec Table 3: "Definition of reserved addresses." Using reserved addresses in tests would cause NACK responses (correct behavior) but confuse the scoreboard (which expects matching behavior). By excluding them, tests only use addresses our slave will respond to.

```
constraint data_size_c {
    num_data >= 0;          // Can be zero bytes (address-only transaction)
    num_data <= 32;         // Max 32 bytes (one I2C page)
    data.size() == num_data; // Queue size must match declared count
}
```

The `data.size() == num_data` constraint is a size constraint. Without it, `num_data` might be 5 but `data` might only have 3 elements — the driver would crash when trying to access `data[3]`.

10.5 The `post_randomize` Fix

The problem: After randomization, `data` queue might contain `X` values if the constraint solver doesn't fully resolve queue contents. SystemVerilog solvers handle queue element initialization differently across tools.

The fix:

```
function void post_randomize();
    // post_randomize() runs automatically AFTER randomize() succeeds
    data = new[num_data];    // Re-allocate with exact size
    foreach (data[i]) begin
        if (data[i] === 8'bx || data[i] === 8'bz) begin
            data[i] = $urandom_range(0, 255);    // Fill any X/Z with random
        end
    end
endfunction
```

`post_randomize()` is a special UVM hook called automatically after successful randomization. Using it here ensures data is always valid, regardless of simulator behavior.

10.6 The Standard UVM Methods

```
virtual function void do_copy(uvm_object rhs);
    i2c_seq_item item;
    super.do_copy(rhs);              // Copy base class fields
    if (!$cast(item, rhs)) return;   // Type-check the cast
    addr = item.addr;
    dir  = item.dir;
    // ... copy all fields ...
endfunction
```

`do_copy` is called by `item.copy()`. The `$cast` checks that `rhs` is actually an `i2c_seq_item` (not just a `uvm_object`). If the cast fails, we return without crashing.

```
virtual function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    i2c_seq_item item;
    if (!$cast(item, rhs)) return 0;
    return (super.do_compare(rhs, comparer) &&
            (addr == item.addr) &&
            (dir  == item.dir) &&
            (data == item.data) &&
            (num_data == item.num_data));
endfunction
```

`do_compare` is called by the scoreboard when comparing expected vs. actual transactions. Returns 1 if equal, 0 if different. Note: `data == item.data` compares two queues element-by-element in SystemVerilog — this works correctly for queues.

---

11. Phase 4 — UVM Driver: `uvm/i2c_master_driver.sv`

11.1 The Driver's Contract

The driver is the bridge between the abstract sequence item world and the physical pin world:

```
Sequence Item                    Physical Bus
(abstract)                       (pin-level)
──────────────────────────────────────────────
addr = 0x50              →    SDA: 1 0 1 0 0 0 0
dir  = WRITE             →    SDA: 0 (R/W bit)
data = [0xA5]            →    SDA: 1 0 1 0 0 1 0 1
start_cond = 1           →    SDA falls while SCL high
stop_cond  = 1           →    SDA rises while SCL high
```

11.2 Getting the Interface

```
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db #(virtual i2c_if)::get(this, "", "i2c_vif", vif)) begin
        `uvm_fatal(get_type_name(), "Virtual interface not found in config_db")
    end
endfunction
```

`uvm_config_db::get` retrieves the interface from the database the testbench set up. The `""` in the second argument means "look in my own context and up the hierarchy." If the database lookup fails, `uvm_fatal` terminates simulation immediately — there's no point continuing without the interface.

11.3 The Run Phase Loop

```
virtual task run_phase(uvm_phase phase);
    i2c_seq_item req_item;

    wait(vif.rst_n === 1'b1);    // Don't drive anything during reset

    forever begin
        seq_item_port.get_next_item(req_item);    // BLOCKING: waits for sequencer

        // Execute the transaction
        if (req_item.start_cond) send_start();
```

```
            send_byte({req_item.addr, req_item.dir[0]});    // Address byte
            wait_for_ack(rsp_item);
            // ... data bytes ...
            if (req_item.stop_cond) send_stop();

            seq_item_port.item_done();    // Signal sequencer: done with this item
        end
    endtask
```

`get_next_item` blocks (suspends the task) until a new sequence item arrives from the sequencer. The driver processes it, drives the bus, then calls `item_done()` to signal the sequencer to send the next item. This is the standard consumer pattern.

11.4 Clock-Edge Based Timing (The Fix)

The broken interface tasks used `#50` delays. These are absolute time delays — fine for a specific simulator setup, but wrong for a proper UVM environment where timing should be based on clock edges.

The fixed driver uses `@(posedge vif.clk)`:

```
virtual task send_start();
    vif.sda <= 1'b1;
    vif.scl <= 1'b1;
    @(posedge vif.clk); @(posedge vif.clk);    // 2 clock cycles setup
    vif.sda <= 1'b0;                            // SDA falls
    @(posedge vif.clk); @(posedge vif.clk);    // 2 clock cycles hold
    vif.scl <= 1'b0;                            // SCL falls
    @(posedge vif.clk);
endtask
```

Why clock-edge based?

1. **Reproducible**: 2 clock cycles is always 2 clock cycles, regardless of simulator time step.
2. **Consistent with DUT**: The DUT is clocked. Making the driver clock-synchronous means driver and DUT stay in lockstep.
3. **Portable**: If the clock frequency changes (say to 100 MHz), the timing automatically adjusts in terms of nanoseconds while keeping the same cycle count.

11.5 NACK for Last Read Byte

```
foreach (req_item.data[i]) begin
    receive_byte(rsp_item);
    if (i < req_item.data.size() - 1) begin
        send_ack();     // ACK: "I want another byte"
```

```
    end else begin
        send_nack();    // NACK: "This is the last byte I want"
    end
end
```

I2C read protocol rule: After each byte the master receives from the slave, the master sends either:

- ACK (SDA = 0): "Keep sending, I want more bytes"
- NACK (SDA = 1): "That's the last byte I want"

Without this distinction, the slave would keep sending data indefinitely after the last byte.

---

12. Phase 4 — UVM Monitor: `uvm/i2c_monitor.sv`

12.1 The Monitor's Contract

The monitor is completely passive:

- It **never drives** any bus signal.
- It **only watches** SDA and SCL.
- It **reconstructs** transactions from the waveforms it observes.
- It **publishes** complete transactions to anyone listening (scoreboard, coverage).

This separation is critical: the monitor must work correctly whether the traffic was generated by the UVM driver, a BFM, a different test tool, or even real hardware. It has no dependency on the stimulus path.

12.2 Two Parallel Threads

```
virtual task run_phase(uvm_phase phase);
    fork
        monitor_bus_events();    // Thread 1: watches for START/STOP
        monitor_data_bits();     // Thread 2: watches for data bits
    join
endtask
```

`fork...join` launches both tasks simultaneously. They run in parallel forever. Thread 1 watches for START/STOP conditions (SDA transitions while SCL high). Thread 2 watches for data bits (SDA stable while SCL rising edge).

Why two threads? The two events have different triggers. Thread 1 triggers on SDA edges with SCL=1. Thread 2 triggers on SCL rising edges. Combining them in one thread would require complex conditionals. Separating them makes each simpler.

12.3 START/STOP Detection (with One-Shot Fix)

```
virtual task monitor_bus_events();
    bit prev_scl, prev_sda;
    prev_scl = 1'b1;
    prev_sda = 1'b1;

    forever begin
        @(negedge vif.scl or negedge vif.sda or posedge vif.sda);
        // Triggers on: SCL falling, SDA falling, or SDA rising

        // START: SDA HIGH→LOW while SCL is HIGH
        if (prev_sda == 1'b1 && vif.sda == 1'b0 && prev_scl == 1'b1) begin
            if (!in_transaction) begin    // ONE-SHOT FIX: only trigger once
                start_detected = 1;
                current_item = new("i2c_item");
                in_transaction = 1;
                bit_cnt = 0;
            end
        end

        // STOP: SDA LOW→HIGH while SCL is HIGH
        if (prev_sda == 1'b0 && vif.sda == 1'b1 && prev_scl == 1'b1) begin
            if (in_transaction) begin    // Only close if we opened a transaction
                finalize_and_write_item();
                in_transaction = 0;
                start_detected = 0;
            end
        end

        prev_scl = vif.scl;
        prev_sda = vif.sda;
    end
endtask
```

The one-shot protection: Without `if (!in_transaction)`, every SDA falling edge while SCL=1 would create a new item. A Repeated START (Sr) would reset the current transaction mid-stream. The guard ensures only one item is active at a time.

12.4 Data Bit Sampling

```
virtual task monitor_data_bits();
    logic prev_scl = 1'b0;
```

```
    forever begin
        @(posedge vif.scl);    // Wait for SCL rising edge

        if (!in_transaction) continue;    // Ignore activity outside transactions

        shift_reg = {shift_reg[6:0], vif.sda};    // Shift in SDA value
        bit_cnt = bit_cnt + 1;

        if (bit_cnt == 8) begin    // Complete byte received
            if (!addr_received) begin
                // First complete byte is the address
                current_item.addr = shift_reg[7:1];
                current_item.dir  = i2c_dir_e'(shift_reg[0]);
                addr_received = 1;
            end else begin
                // Subsequent bytes are data
                current_item.data.push_back(shift_reg);
                current_item.num_data++;
            end
            bit_cnt = 0;
            shift_reg = 8'h00;
        end
        // Note: bit_cnt == 9 would be the ACK bit — we skip it
    end
endtask
```

Sampling on rising SCL edge: This matches the I2C spec exactly. The spec says "data on SDA is valid during the HIGH period of SCL." The rising edge is the start of the HIGH period — the earliest legally valid sample point.

Skipping the ACK bit: The monitor counts up to 8 bits, processes the byte, resets, and waits for the next 8 bits. It doesn't try to capture the 9th bit (ACK). This is intentional — the ACK is a bus control signal, not application data.

---

13. Phase 4 — UVM Scoreboard: `uvm/i2c_scoreboard.sv`

13.1 The Scoreboard's Contract

The scoreboard is the automated checker — it determines PASS or FAIL for every transaction. It has two sides:

1. **Expected side**: What SHOULD happen (from the sequence/driver)
2. **Actual side**: What DID happen (from the monitor)

It compares them and raises errors on mismatches.

13.2 The Reference Model

The scoreboard's reference model is a **shadow** of the slave's internal register bank:

```
logic [7:0] ref_reg_bank [0:7];    // Shadow of slave's 8 registers
```

When the driver sends a WRITE transaction to the slave, the scoreboard also updates `ref_reg_bank`. When the driver sends a READ transaction, the scoreboard knows what value to expect — it reads `ref_reg_bank` to predict what the slave will return.

This is more powerful than "check that SDA matches the expected bit pattern." It checks semantic correctness: "After writing 0xA5 to register 2, reading register 2 must return 0xA5."

13.3 The TLM Wiring Bug and Fix

The broken version:

```
uvm_analysis_imp #(i2c_seq_item, i2c_scoreboard) expected_items_export;
uvm_analysis_imp #(i2c_seq_item, i2c_scoreboard) actual_items_export;
// Both resolve to void write(i2c_seq_item t) — same name, same class → CONFLICT
```

Why this fails: `uvm_analysis_imp` generates a `write()` function in the parent class. If you have two `uvm_analysis_imp` in the same class, they both try to generate `write()` — duplicate definition error.

The fix using macros:

```
`uvm_analysis_imp_decl(_expected)   // Creates write_expected() and imp type
`uvm_analysis_imp_decl(_actual)     // Creates write_actual() and imp type

class i2c_scoreboard extends uvm_scoreboard;
    uvm_analysis_imp_expected #(i2c_seq_item, i2c_scoreboard) expected_export;
    uvm_analysis_imp_actual   #(i2c_seq_item, i2c_scoreboard) actual_export;

    function void write_expected(i2c_seq_item item);
        update_ref_model(item);    // Update shadow register bank
    endfunction

    function void write_actual(i2c_seq_item item);
        check_transaction(item);   // Compare with reference model
    endfunction
endclass
```

The macros generate specialized imp classes with different `write` function names. Now the two exports call different functions — no conflict.

---

14. Phase 6 — SVA Assertions: `sva/i2c_protocol_assertions.sv`

14.1 What SVA Assertions Do

SVA (SystemVerilog Assertions) are **temporal property checkers**. They run automatically on every clock cycle and check whether the waveforms obey the I2C protocol.

Unlike a scoreboard (which checks transaction semantics), assertions check **bus protocol timing**:

- Does SDA remain stable while SCL is HIGH?
- Does a START condition have the right SDA/SCL relationship?
- Does the bus return to idle after STOP?

14.2 The Assertion Structure

```
property p_some_property;
    @(posedge clk)          // Checked on every rising clock edge
    disable iff (!rst_n)    // Don't check during reset
    antecedent              // IF this condition is true...
    |->                     // THEN (same cycle) or |=> (next cycle)
    consequent;             // ...this must be true
endproperty

assert property (p_some_property)
    else `uvm_error("LABEL", "Failure message");
```

`|->` vs `|=>`:

- `|->` (overlapping implication): consequent checked on the SAME clock cycle as antecedent.
- `|=>` (non-overlapping implication): consequent checked on the NEXT clock cycle.

14.3 A1: START Condition

```
property p_start_cond;
    @(posedge clk)
    disable iff (!rst_n)
    (scl_samp == 1'b1 && sda_fall)    // SCL is HIGH and SDA just fell
```

```
    |=>                                  // On the NEXT cycle:
    (scl_samp == 1'b1);                 // SCL must still be HIGH
endproperty
```

What this catches: If someone tries to generate a START condition but SCL is LOW (just a normal data bit change), or if SCL falls at the same time as SDA (timing violation), this assertion fires.

The `sda_fall` signal:

```
assign sda_fall = (sda_d == 1'b1 && sda_samp == 1'b0);  // SDA was HIGH, now LOW
```

This is the same edge detection pattern used in the slave RTL.

14.4 A3: SDA Stability (The Vacuous Truth Fix)

The broken version:

```
(scl_samp == 1'b1) |-> $stable(sda_samp) [*0:$];
//                                        ↑
//                                Matches 0 or more times
//                                0 times ALWAYS matches → vacuously true
```

Why `[*0:$]` is wrong: The `[*0:$]` repetition means "zero or more consecutive cycles." Zero repetitions always satisfy the property regardless of what actually happens. This assertion never fails — it's useless.

The fixed version:

```
property p_sda_stable_scl_high;
    @(posedge clk)
    disable iff (!rst_n)
    (in_data_phase && scl_samp == 1'b1) |-> $stable(sda_samp);
endproperty
```

`$stable(signal)` means "signal did not change between the previous clock edge and this one." Combined with `in_data_phase` (not during START/STOP), this correctly checks that SDA is stable on every cycle where SCL is HIGH during a data phase.

14.5 A4: ACK Timing

```
property p_ack_low;
    @(posedge clk)
    disable iff (!rst_n)
```

```
    (in_ack_phase && scl_samp == 1'b1)    // During 9th SCL HIGH period
    |->
    (##[0:1] (sda_samp == 1'b0));         // SDA must be LOW within 0–1 cycles
endproperty
```

The bit counter for ACK detection:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        bit_counter   <= 4'd0;
        in_data_phase <= 1'b0;
        in_ack_phase  <= 1'b0;
    end else begin
        if (start_cond_asserted) begin
            bit_counter   <= 4'd0;
            in_data_phase <= 1'b1;
            in_ack_phase  <= 1'b0;
        end else if (in_data_phase && scl_fall) begin
            bit_counter <= bit_counter + 1;
            if (bit_counter == 4'd7) begin
                in_data_phase <= 1'b0;
                in_ack_phase  <= 1'b1;    // 8th bit done → enter ACK phase
            end
        end else if (in_ack_phase && scl_fall) begin
            in_ack_phase  <= 1'b0;
            in_data_phase <= 1'b1;        // ACK done → next data byte
            bit_counter   <= 4'd0;
        end
    end
end
```

This counter runs inside the assertion module and tracks which bit of the transaction we're on. When `in_ack_phase` is true, the assertion checks that SDA is pulled LOW.

14.6 A6: Clock Stretch Bounded

```
property p_clk_stretch_bounded;
    @(posedge clk)
    disable iff (!rst_n)
    (scl_samp == 1'b0 && scl_d == 1'b1)         // SCL just fell
    |->
    (##[1:MAX_CLK_STRETCH] (scl_samp == 1'b1));  // Must rise within MAX_CLK_STRETCH
endproperty
```

`##[1:MAX_CLK_STRETCH]` is a temporal range: "within 1 to 1000 clock cycles." If SCL falls and doesn't rise within 1000 cycles, the assertion fires. This catches stuck-low buses and slaves that stretch indefinitely.

15. The Complete Bug Fix Log — What Went Wrong and Why

15.1 Why These Specific Bugs Occurred

Understanding why a bug was made is more valuable than just knowing the fix. These are the mental mistakes behind each bug:

CRITICAL #1 — Broken Master Wrapper

Mental mistake: Confusion between module port declaration syntax and module instantiation syntax. Port declarations are just signals — they live in the module header. Instantiations are structural — they live in the module body. **Lesson:** Port lists declare the module's I/O. Module body contains logic and sub-module instances. These are in different syntactic locations.

CRITICAL #2 — SCL Permanently LOW

Mental mistake: Copy-paste error. Wrote the right structure (ternary `? :`) but put `1'b0` in both branches while intending `1'b0 : 1'b1`. **Lesson:** After writing any logic that controls a bus signal, immediately verify: "Is this signal ever different from its reset/idle value?" If SCL is always 0 in TX_BIT, that's a red flag that catches itself during first waveform inspection.

CRITICAL #3 — next_state Multi-Driver

Mental mistake: Thinking procedurally (like C), where you can modify a variable from anywhere. In hardware, a signal has exactly one driver. `always_comb` drives `next_state`. `always_ff` cannot also drive it. **Lesson:** In synthesizable RTL, every net has exactly one driver (except tri-state buses). If you catch yourself writing to a signal in two different always blocks, it's wrong. The exception is reset logic in `always_ff` — but that's for the registered state, not the combinational next-state.

CRITICAL #4 — Slave Register Write Captures ACK Bit

Mental mistake: Off-by-one in state machine timing. The code intended to capture the 8th data bit but executed one clock cycle too late, during the 9th clock (ACK period). **Lesson:** Draw the state machine transition diagram with explicit cycle counts. Label exactly when each signal is valid. The 8th data bit is valid during cycles 1-8. The ACK bit is cycle 9. The state machine transitions on cycle 9 — so any logic in the "entering ACK state" context sees cycle 9 data.

CRITICAL #5 — Floating Inputs (stretch_en, stretch_cycles)

Mental mistake: "I'll connect it later" or "it's unused so it doesn't matter." In simulation, unconnected inputs are `Z`, which propagates as `X`. **Lesson:** Every port must be connected. During early development, connect unused inputs to `'0`. Never leave ports unconnected in a testbench.

MAJOR #1 — Timer Infinite Loop

Mental mistake: The original timer used `timer_en` as a flag and cleared itself on expiry. But `timer_en` was always 1, so the timer would reset and expire again and again, never staying in a state long enough. **Lesson:** Timer design requires clear thinking about: when does the timer start? when does it reset? what does expiry mean? The fixed design uses `timer_clr = (state != next_state)` — clean and correct.

MAJOR #3 — Slave TX Drives 1'bz

Mental mistake: Confusion about who drives SDA in read mode. The programmer wrote "release SDA" thinking the slave should release it, forgetting that in read mode the slave IS the data source. **Lesson:** Before writing any bus-driving logic, explicitly answer: "At this moment in time, WHO owns SDA?" If the answer is "the slave," the slave must drive data, not release.

---

16. How Everything Connects — Full Data Flow

16.1 The Complete Simulation Flow

```
START OF SIMULATION
       |
       ▼
top_tb.sv: Generate clk, rst_n
       |
       ▼
top_tb.sv: uvm_config_db::set(i2c_bus_if) → store interface
       |
       ▼
top_tb.sv: run_test() → instantiate test from +UVM_TESTNAME
       |
       ▼
Test.build_phase() → creates env → env creates agent → agent creates driver+monitor
       |
       ▼
All components: build_phase → get virtual interface from config_db
       |
       ▼
```

```
UVM run_phase starts:
        │
        ├──→ Test: Start sequence on sequencer
        │          │
        │          ▼
        │    Sequencer: receive items from sequence, forward to driver
        │          │
        │          ▼
        │    Driver: get_next_item() → translate to pin activity
        │          │
        │          ▼
        │    Bus: SDA and SCL toggle according to I2C protocol
        │          │
        │          ├──→ Master DUT: sees SCL/SDA → processes command
        │          │
        │          └──→ Slave DUT: sees SCL/SDA → receives/transmits data
        │
        ├──→ Monitor (parallel): watching SDA/SCL
        │          │
        │          ▼
        │    Monitor: detect START → sample bits → detect STOP
        │    Monitor: publish completed transaction to analysis port
        │          │
        │          ├──→ Scoreboard: compare with reference model → PASS/FAIL
        │          │
        │          └──→ Coverage: sample covergroups → update coverage %
        │
        └──→ SVA Assertions (continuous): check protocol rules every clock cycle

END OF TEST: report_phase prints scoreboard results and coverage summary
```

16.2 A Single Write Transaction, Step by Step

Scenario: Write the value `0xA5` to slave address `0x50`.

```
Step 1: Test creates sequence item
──────────────────────────────────
item.addr       = 7'h50
item.dir        = DIR_WRITE
item.data       = {8'hA5}
item.start_cond = 1
item.stop_cond  = 1

Step 2: Driver gets item, drives START
──────────────────────────────────────
SDA: 1→0 (while SCL=1)    ← START condition
SCL: 1→0

Assertion A1: checks SDA fell while SCL=1 ✓
```

```
Step 3: Driver sends address byte 0xA0 (= 0x50<<1 | 0)
──────────────────────────────────────────────────────

Bit 7: SDA=1 (0xA0 = 1010_0000, bit 7 is 1)
Bit 6: SDA=0
Bit 5: SDA=1
Bit 4: SDA=0
Bit 3: SDA=0
Bit 2: SDA=0
Bit 1: SDA=0
Bit 0: SDA=0 (R/W=0 means write)

For each bit: SCL goes low, driver sets SDA, SCL goes high, slave samples
Assertion A3: checks SDA stable while SCL=1 on each cycle ✓

Monitor: detects SCL rising edges, samples SDA, accumulates in shift_reg

Step 4: Master releases SDA for 9th clock (ACK)
──────────────────────────────────────────────

Master: sda_o = 1'b1 (release)
Slave:  addr_matched = (0x50 == SLAVE_ADDR) → TRUE
Slave:  sda_o = 1'b0 (pull ACK low)
Bus SDA: 0 (slave wins on wired-AND)

Assertion A4: checks SDA=0 during 9th SCL high ✓

Master samples: ack_received = 1 ✓

Monitor: sees 8 bits accumulated → first byte = address.
         Stores: current_item.addr = 0x50, current_item.dir = WRITE

Step 5: Driver sends data byte 0xA5
──────────────────────────────────

Same bit-by-bit process, 8 bits
0xA5 = 1010_0101

Step 6: Slave ACKs the data byte
────────────────────────────────

Slave sees 8 bits received → stores in latched_rx_byte = 0xA5
Slave: internal_regs[0] = 0xA5
Slave pulls SDA low → ACK

Monitor: captures 0xA5, pushes to current_item.data

Step 7: Driver drives STOP
──────────────────────────

SDA: 0→1 while SCL=1
Assertion A2: checks SDA rose while SCL=1 ✓
Assertion A5: checks bus goes free after STOP ✓

Monitor: detects STOP → calls finalize_and_write_item()
Monitor publishes: {addr=0x50, dir=WRITE, data=[0xA5]}

Step 8: Scoreboard receives published item
```

```
──────────────────────────────────────────
Scoreboard.write_actual({addr=0x50, dir=WRITE, data=[0xA5]})
ref_reg_bank[0] is checked:
  Expected: 0xA5
  Actual:   Scoreboard reads slave DUT's reg_bank[0] = 0xA5
  Result:   MATCH → match_count++ → no error
```

---

17. Interview Preparation — Questions This Project Answers

17.1 Protocol Questions

Q: Explain the I2C protocol timing.

A: I2C uses two open-drain lines, SDA and SCL. The master generates SCL. Data is sent MSB first, 8 bits per byte. SDA must be stable while SCL is HIGH — changes only happen when SCL is LOW. A START condition is SDA falling while SCL is HIGH. A STOP condition is SDA rising while SCL is HIGH. After every 8-bit byte, the 9th clock cycle is the ACK/NACK cycle where the receiver acknowledges by pulling SDA LOW.

Follow-up: What is clock stretching?

A slave can hold SCL LOW to pause the master while it prepares data. The master must check SCL after releasing it — if SCL is still LOW, it must wait. This is clock stretching. I implemented a MAX_CLK_STRETCH = 1000 cycle bound in the master, after which a timeout is flagged.

Q: How did you handle the bidirectional SDA line in your RTL?

A: I used the open-drain tri-state model. Internally, I have `sda_o` (what I want to drive) and `sda_i` (what's on the bus). The assignment `assign sda = (sda_o == 1'b0) ? 1'b0 : 1'bz` means I only pull LOW actively; releasing the line (`1'bz`) lets the pull-up resistor or the other device determine the value. I read back `sda_i = sda` for ACK detection and arbitration.

17.2 Verification Questions

Q: What was your scoreboard's reference model?

A: A shadow register bank — 8 registers that mirror the slave's internal state. When the driver sends a WRITE transaction, the scoreboard updates the shadow bank. When a READ transaction occurs, the scoreboard predicts the expected read data from the shadow bank and compares it against what the monitor observed on the bus. This catches not just signal-level errors but semantic errors — if the slave returns wrong data even though all bus signals look correct.

Q: How did you achieve coverage closure?

A: I ran `test_random_traffic` with 1000 transactions first to hit most bins. Then I inspected the coverage report to find uncovered bins. Walk-1 data patterns and clock-stretch scenarios were uncovered, so I added directed tests targeting those specifically. I also needed explicit tests for NACK-on-address and Repeated START to cover the `cg_protocol` and `cg_error` covergroups.

Q: What was the most interesting bug you found?

A: The slave register write was capturing the ACK bit as the 8th data bit. The register bank would store a value with the LSB always equal to the ACK bit (0 for ACK, 1 for NACK). A write of 0xA4 (1010_0100) would be stored as 0xA4 if ACKed (correct) but 0xA5 (1010_0101) if NACKed — the stored value depended on bus conditions, not just the data sent. The fix was to latch the byte at the correct moment (8th data bit's falling SCL edge), not at the transition into the ACK state.

Q: Explain your UVM architecture.

A: I have a master agent (active: driver + monitor + sequencer) that drives transactions to the DUT master. The monitor passively reconstructs transactions from the bus waveforms and publishes them via TLM analysis ports to the scoreboard and coverage collector. The scoreboard has a shadow reference model. The environment also contains 7 SVA assertions bound to the DUT that check protocol timing continuously.

Q: What is the difference between `|->` and `|=>` in SVA?

A: `|->` is overlapping implication — the consequent is evaluated starting from the SAME cycle as the antecedent. `|=>` is non-overlapping — the consequent is evaluated starting from the NEXT cycle. For example, my START assertion uses `|=>` because after SDA falls while SCL=1, the requirement is that SCL REMAINS high on the following cycle — a next-cycle check.

Q: How does the UVM factory work and why is it useful?

A: The UVM factory is a global registry where classes register themselves with `uvm_component_utils` or `uvm_object_utils`. When you create an object with `TypeName::type_id::create(...)`, the factory looks up if any override has been registered for that type. If you register `i2c_error_seq_item` as an override for `i2c_seq_item`, the factory substitutes it everywhere without any test code changes. This is how you create a test that injects errors just by setting a factory override, reusing all the same sequences and environment.

This document covers every architectural decision, every file, every line of code, and every bug found in the I2C UVM project. The depth here represents what an engineer actually thinks while building industrial-grade verification from scratch. Every `if`, every `typedef`, every task name has a reason — and now you know all of them.

Document end.

Project: I2C Protocol RTL Design + Full UVM Verification
Author: Kushal Pitaliya | CHARUSAT University ECE
Reference: NXP UM10204 Rev 7.0, IEEE 1800-2017, UVM 1.2